INF398 - Introducción al aprendizaje automático

Linear models for regression

Alejandro Veloz

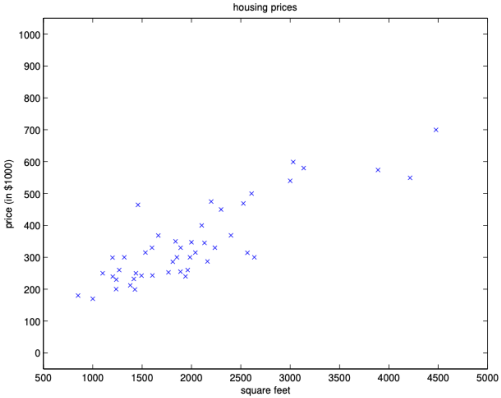
Overview

1. Problem formulation.
2. Parameters estimation: Maximum Likelihood (ML).
3. Parameters estimation: Maximum A Posteriori.
4. Bayesian linear regression.
5. Maximum Likelihood as Orthogonal Projection.

Regression problem

Suppose we have a dataset giving the living areas and prices of houses:

Living area (feet ^{2})	Price ($1000\$$ s)
2104	400
1600	330
2400	369
1416	232
3000	540
$\vdots$	$\vdots$



Regression problem - notation

$x^{(i)}$ to denote the “input” variables (living area in this example), also called **input features.**

Regression problem - notation

$x^{(i)}$ to denote the “input” variables (living area in this example), also called **input features.**

$y^{(i)}$ to denote the “output” or **target variable** that we are trying to predict (price).

Regression problem - notation

$x^{(i)}$ to denote the “input” variables (living area in this example), also called **input features.**

$y^{(i)}$ to denote the “output” or **target variable** that we are trying to predict (price).

A pair $\left(x^{(i)}, y^{(i)}\right)$ is called a **training example.**

Regression problem - notation

$x^{(i)}$ to denote the "input" variables (living area in this example), also called **input features.**

$y^{(i)}$ to denote the "output" or **target variable** that we are trying to predict (price).

A pair $\left(x^{(i)}, y^{(i)}\right)$ is called a **training example.**

The **training set** is the list of n training examples:

$$\left\{\left(x^{(i)}, y^{(i)}\right); i = 1, \ldots, n\right\}.$$

Regression problem - notation

$x^{(i)}$ to denote the "input" variables (living area in this example), also called **input features.**

$y^{(i)}$ to denote the "output" or **target variable** that we are trying to predict (price).

A pair $\left(x^{(i)}, y^{(i)}\right)$ is called a **training example.**

The **training set** is the list of n training examples:

$$\left\{\left(x^{(i)}, y^{(i)}\right); i = 1, \ldots, n\right\}.$$

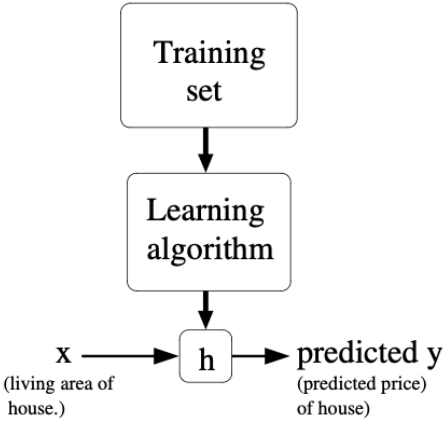
We will also use $\mathcal{X}$ to denote the space of input values, and $\mathcal{Y}$ the space of output values. In this example, $\mathcal{X} = \mathcal{Y} = \mathbb{R}$.

Regression problem

Regression Goal: Learn a function $h : \mathcal{X} \to \mathcal{Y}$ that predicts one or more continuous target variables y from a D-dimensional input vector $\mathbf{x}$, such that $h(x)$ provides a good approximation of the true output.

This function h is called **hypothesis**.

Regression problem. The target variable is continuous.



Linear regression

We must decide how to represent functions/hypotheses h.

We decide to approximate y as a linear function of x:

$$h_\theta(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

θ_i 's are the parameters (also called weights) parameterizing the space of linear functions mapping from $\mathcal{X}$ to $\mathcal{Y}$.

We use the convention $x_0 = 1$ (intercept term) to write:

$$h(\mathbf{x}) = \sum_{i=0}^{D} \theta_i x_i = \boldsymbol{\theta}^\top \mathbf{x}$$

Loss function

Given a training set, how do we learn the parameters θ?

Loss function

Given a training set, how do we learn the parameters θ?

One reasonable method seems to be to make $h(\mathbf{x})$ close to y, at least for the training examples we have.

Loss function

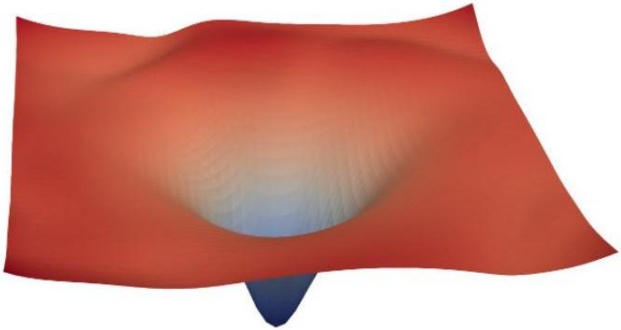
Given a training set, how do we learn the parameters θ?

One reasonable method seems to be to make $h(\mathbf{x})$ close to y, at least for the training examples we have.

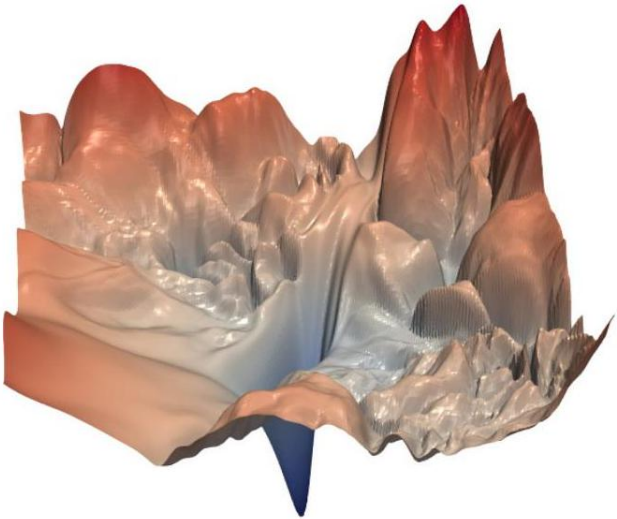
We define the **cost function** to measures, for each value of the θ's, how close the $h\left(\mathbf{x}^{(i)}\right)$'s are to the corresponding $y^{(i)}$'s:

$$J(\boldsymbol{\theta}) = \frac{1}{2}\sum_{i=1}^{n}\left(h_\theta\left(\mathbf{x}^{(i)}\right) - y^{(i)}\right)^2$$

Error surface



Error surface



Learning algorithm

Random-Regression

Require: Data $\mathcal{D}$, integer k

for $i = 1$ **to** k **do**
  Randomly generate hypothesis $\boldsymbol{\theta}(i)$
end for
Let $i = \arg\min_j J(\boldsymbol{\theta}(j); \mathcal{D})$
return $\boldsymbol{\theta}(i)$

Learning algorithm: sequential learning

We want to choose $\boldsymbol{\theta}$ so as to minimize $J(\boldsymbol{\theta})$.

Learning algorithm: sequential learning

We want to choose $\boldsymbol{\theta}$ so as to minimize $J(\boldsymbol{\theta})$.

Search algorithm (gradient descent): Start with some "initial guess" for $\boldsymbol{\theta}$, and repeatedly change $\boldsymbol{\theta}$ to make $J(\boldsymbol{\theta})$ smaller, until hopefully we converge to a value of $\boldsymbol{\theta}$ that minimizes $J(\boldsymbol{\theta})$.

Learning algorithm: sequential learning

We want to choose $\boldsymbol{\theta}$ so as to minimize $J(\boldsymbol{\theta})$.

Search algorithm (gradient descent): Start with some "initial guess" for $\boldsymbol{\theta}$, and repeatedly change $\boldsymbol{\theta}$ to make $J(\boldsymbol{\theta})$ smaller, until hopefully we converge to a value of $\boldsymbol{\theta}$ that minimizes $J(\boldsymbol{\theta})$.

The update step is:

$$\boldsymbol{\theta}^{(\tau+1)} = \boldsymbol{\theta}^{(\tau)} - \eta \nabla E_n$$

where τ denotes the iteration number, and η is a learning rate parameter.

The value of η needs to be chosen with care to ensure that the algorithm converges (Bishop and Nabney, 2008).

Learning algorithm: sequential learning

For the model:

$$h_\theta(\mathbf{x}) = \theta_0 + \theta_1 x_1 + \ldots + \theta_n x_n \ ,$$

the update step is:

$$\theta_j := \theta_j - \eta \frac{\partial}{\partial \theta_j} J(\theta).$$

This update is simultaneously performed for all values of $j = 0, \ldots, d$.

η is called the learning rate.

Learning algorithm: sequential learning

For the linear regression model, the update step can be:

Batch gradient descent

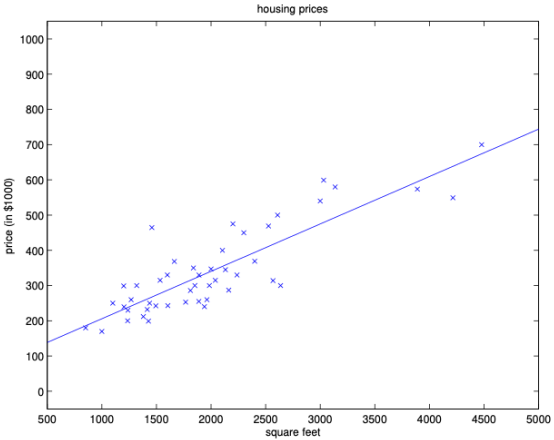
$$\theta_j := \theta_j + \eta \sum_{i=1}^{n} \left(y^{(i)} - h_\theta\left(\mathbf{x}^{(i)}\right)\right) x_j^{(i)}, \text{(for every } j\text{)}$$

Stochastic or incremental gradient descent

$$\theta_j := \theta_j + \eta \left(y^{(i)} - h_\theta\left(\mathbf{x}^{(i)}\right)\right) x_j^{(i)}, \quad \text{(for every } i, j\text{)}$$

$$\boldsymbol{\theta} := \boldsymbol{\theta} + \eta \left(y^{(i)} - h_\theta\left(\mathbf{x}^{(i)}\right)\right) \mathbf{x}^{(i)}, \quad \text{(for every } i\text{)}$$

Regression curve



Linear Basis Function Models

Linear Regression Models. Linear functions of the adjustable parameters:

$$y(\mathbf{x}, \boldsymbol{\theta}) = \theta_0 + \theta_1 x_1 + ... + \theta_D x_D$$

Linear Basis Function Models

Linear Regression Models. Linear functions of the adjustable parameters:

$$y(\mathbf{x}, \boldsymbol{\theta}) = \theta_0 + \theta_1 x_1 + \ldots + \theta_D x_D$$

Linear Basis Function Models. Can be extended to include nonlinear functions of the input variables using basis functions:

$$y(\mathbf{x}, \boldsymbol{\theta}) = \theta_0 + \sum_{j=1}^{M-1} \theta_j \phi_j(\mathbf{x}) = \sum_{j=0}^{M} \theta_j \phi_j(\mathbf{x}) = \boldsymbol{\theta}^\mathsf{T} \boldsymbol{\phi}(\mathbf{x})$$

Linear Basis Function Models

Linear Regression Models. Linear functions of the adjustable parameters:

$$y(\mathbf{x}, \boldsymbol{\theta}) = \theta_0 + \theta_1 x_1 + \ldots + \theta_D x_D$$

Linear Basis Function Models. Can be extended to include nonlinear functions of the input variables using basis functions:

$$y(\mathbf{x}, \boldsymbol{\theta}) = \theta_0 + \sum_{j=1}^{M-1} \theta_j \phi_j(\mathbf{x}) = \sum_{j=0}^{M} \theta_j \phi_j(\mathbf{x}) = \boldsymbol{\theta}^\mathsf{T} \boldsymbol{\phi}(\mathbf{x})$$

Basis Functions. Examples include polynomials, Gaussians, and sigmoidals.

Basis Functions

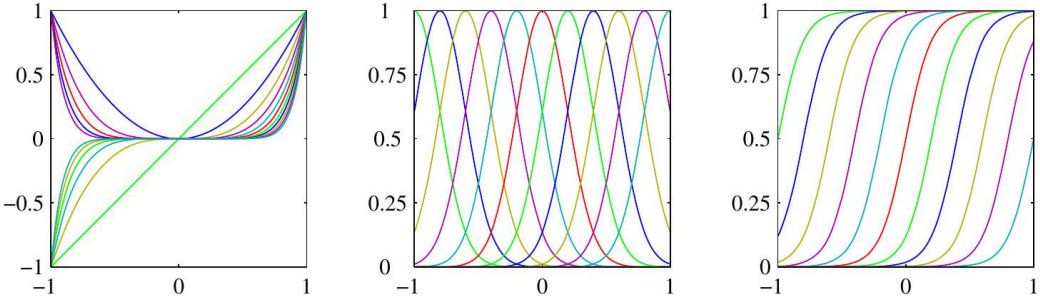
Examples of Basis Functions.

$$\phi_j(x) = x^j \quad \text{(Polynomial)}$$

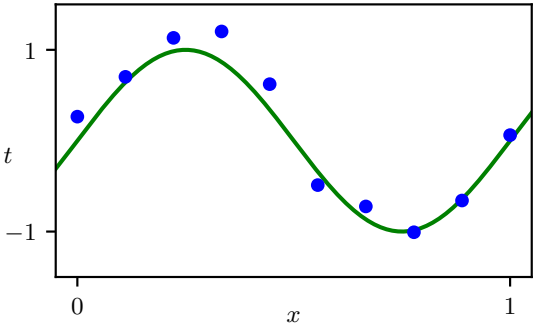
$$\phi_j(x) = \exp\left\{-\frac{(x-\mu_j)^2}{2s^2}\right\} \quad \text{(Gaussian)}$$

$$\phi_j(x) = \sigma\left(\frac{x-\mu_j}{s}\right) \quad \text{(Sigmoidal)}$$

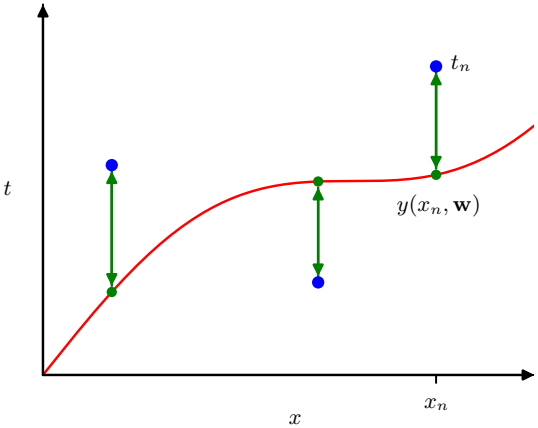
Basis Functions



Example polynomial regression (Bishop 2006)



Example polynomial regression (Bishop 2006)



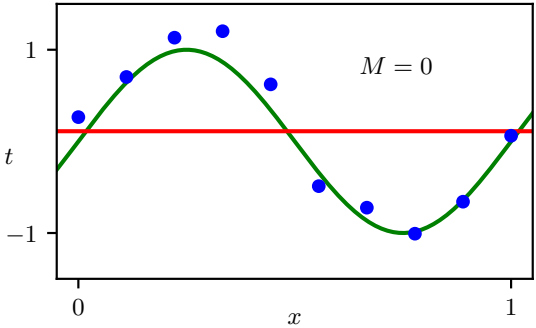
Example polynomial regression (Bishop 2006)

$$h(x, w) = w_0 + w_1 x + w_2 x^2 + \ldots + w_M x^M$$

$$E(w) = \sum_{n=1}^{N} \left(h\left(x_n, w\right) - t_n\right)^2$$

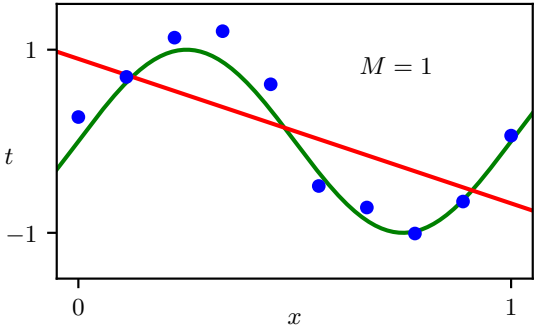
Example polynomial regression (Bishop 2006)

$$h(x, w) = w_0 + w_1 x + w_2 x^2 + ... + w_M x^M$$



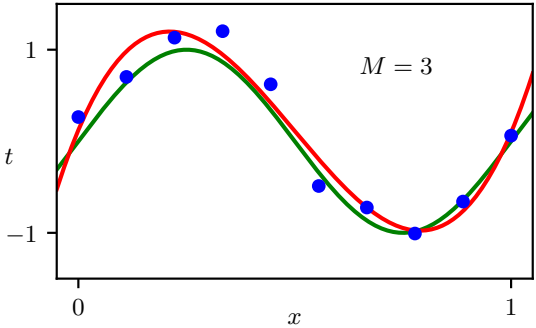
Example polynomial regression (Bishop 2006)

$$h(x, w) = w_0 + w_1 x + w_2 x^2 + ... + w_M x^M$$



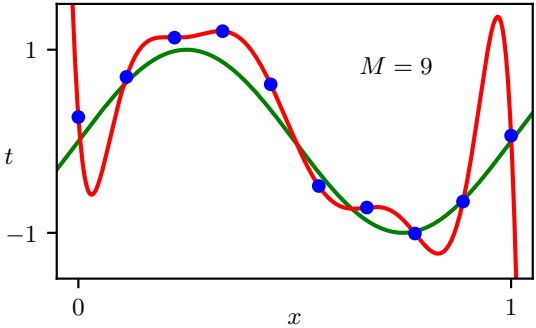
Example polynomial regression (Bishop 2006)

$$h(x, w) = w_0 + w_1 x + w_2 x^2 + ... + w_M x^M$$

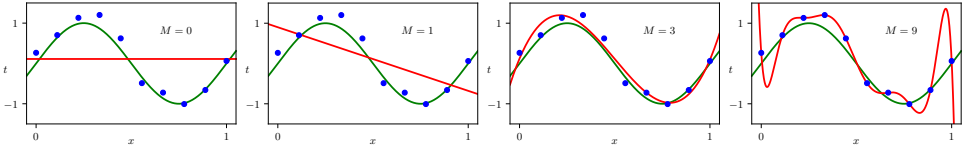


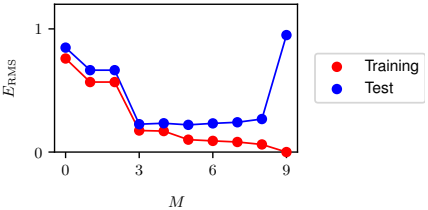
Example polynomial regression (Bishop 2006)

$$h(x, w) = w_0 + w_1 x + w_2 x^2 + ... + w_M x^M$$



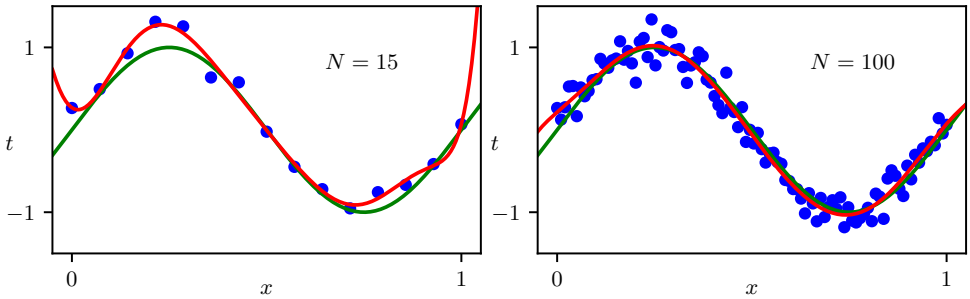
Example polynomial regression (Bishop 2006)





Higher polynomial degree is better (training error always decreases). Test error increases after some polynomial degree.

Example polynomial regression (Bishop 2006)



Regularized least squares

$$E_D(\boldsymbol{\theta}) + \lambda E_W(\boldsymbol{\theta})$$

where λ is the regularization coefficient that controls the relative importance of the data-dependent error $E_D(\boldsymbol{\theta})$ and the regularization term $E_W(\boldsymbol{\theta})$.

Regularized least squares

$$E_D(\boldsymbol{\theta}) + \lambda E_W(\boldsymbol{\theta})$$

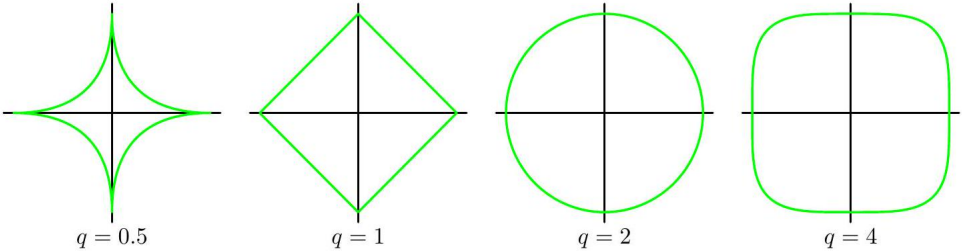
where λ is the regularization coefficient that controls the relative importance of the data-dependent error $E_D(\boldsymbol{\theta})$ and the regularization term $E_W(\boldsymbol{\theta})$.

Common regularizers:

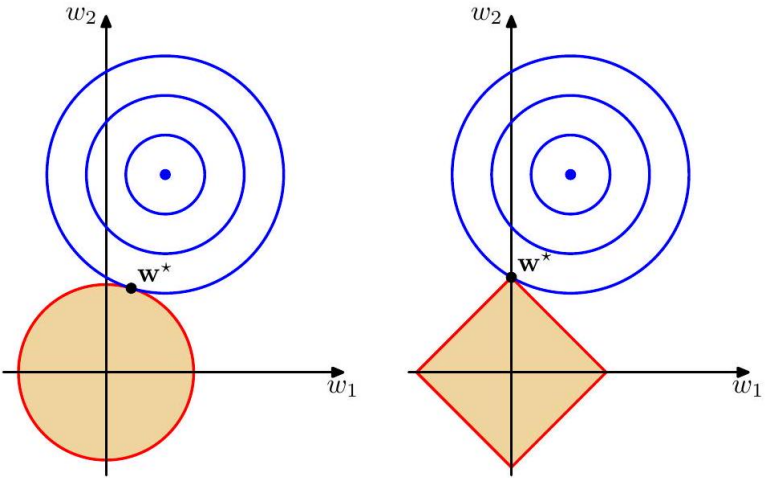
(ridge) $\frac{1}{2}\|\boldsymbol{\theta}\|_2^2 = \frac{1}{2}\boldsymbol{\theta}^\mathsf{T}\boldsymbol{\theta}$

(lasso) $\|\boldsymbol{\theta}\|_1$

Regularization



Regularization



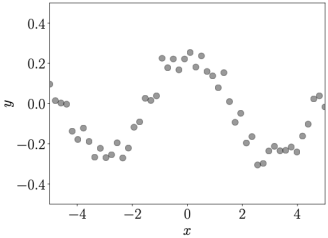
Ridge regression

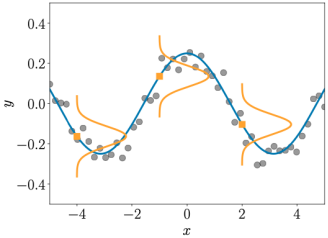
$$\frac{1}{2}\sum_{n=1}^{N}\left\{y_n - \boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{\phi}\left(\mathbf{x}_n\right)\right\}^2 + \frac{\lambda}{2}\boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{\theta}$$

Overview

1. Problem formulation.
2. Parameters estimation: Maximum Likelihood (ML).
3. Parameters estimation: Maximum A Posteriori.
4. Bayesian linear regression.
5. Maximum Likelihood as Orthogonal Projection.

Regression problem





For some input values x_n, we observe noisy function values $y_n = f(x_n) + \epsilon$.

Goal: infer the function f that generalizes well to function values at new inputs.

Regression problem

$$p(y|\mathbf{x}) = p_{Y|\mathbf{x}}(y|\mathbf{x}), \quad Y \sim \mathcal{N}(\mu, \sigma^2) \xrightarrow{\text{simplifies}} \mathcal{N}(y \mid f(\mathbf{x}), \sigma^2)$$

Assume: linear regression, Gaussian noise.

$y = f(\mathbf{x}) + \epsilon$, where $\epsilon \sim \mathcal{N}(0, \sigma^2)$

Likelihood: for $\mathbf{x} \in \mathbb{R}^D$ and $y \in \mathbb{R}$, $p(y \mid \mathbf{x}) = \mathcal{N}(y \mid f(\mathbf{x}), \sigma^2)$.

Linear regression with the parameter $\boldsymbol{\theta} \in \mathbb{R}^D$, i.e., $f(\mathbf{x}) = \mathbf{x}^\top \boldsymbol{\theta}$

$$\boxed{p(y \mid \mathbf{x}) = \mathcal{N}(y \mid \mathbf{x}^\top \boldsymbol{\theta}, \sigma^2) \iff y = \mathbf{x}^\top \boldsymbol{\theta} + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)}$$

Parameters estimation

Training set $\mathcal{D} = \{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$.

Assuming iid N data samples, the likelihood is factorized into:

$$p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta}) = \prod_{n=1}^{N} p(y_n \mid \mathbf{x}_n, \boldsymbol{\theta}) = \prod_{n=1}^{N} \mathcal{N}(y_n \mid \mathbf{x}_n^\intercal, \sigma^2),$$

where $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ and $\mathcal{Y} = \{y_1, \dots, y_n\}$.

Estimation methods: ML and MAP.

MLE (Maximum Likelihood Estimation)

$$\boldsymbol{\theta}_{\text{ML}} = \arg\max_{\boldsymbol{\theta}} p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta}) = \arg\min_{\boldsymbol{\theta}} \Big(-\log p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta}) \Big)$$

For Gaussian noise with $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\mathsf{T}$ and $\mathbf{y} = [y_1, \dots, y_n]^\mathsf{T}$,

$$-\log p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta}) = -\log \prod_{n=1}^{N} p(y_n \mid \mathbf{x}_n, \boldsymbol{\theta}) = -\sum_{n=1}^{N} \log p(y_n \mid \mathbf{x}_n, \boldsymbol{\theta})$$

$$= \frac{1}{2\sigma^2} \sum_{n=1}^{N} (y_n - \mathbf{x}_n^\mathsf{T} \boldsymbol{\theta})^2 + \text{const} = \frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 + \text{const}$$

Negative-log likelihood for $f(\mathbf{x}) = \mathbf{x}^\mathsf{T}\boldsymbol{\theta} + \mathcal{N}(0, \sigma^2)$:

$$-\log p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta}) = \frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2 + \text{const}$$

MLE (Maximum Likelihood Estimation)

For Gaussian noise with $\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_n]^\mathsf{T}$ and $\mathbf{y} = [y_1, \dots, y_n]^\mathsf{T}$,

$$\boldsymbol{\theta}_{\mathsf{ML}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}\boldsymbol{\theta}\|^2.$$

In case of Gaussian noise, $\boldsymbol{\theta}_{\mathsf{ML}} = \boldsymbol{\theta}$ that minimizes the empirical risk with the squared loss function.

Models as functions $=$ Model as probabilistic models.

MLE (Maximum Likelihood Estimation)

We find $\boldsymbol{\theta}$ such that $\frac{\mathrm{d}L}{\mathrm{d}\boldsymbol{\theta}} = 0$

$$\frac{\mathrm{d}L}{\mathrm{d}\boldsymbol{\theta}} = \frac{1}{2\sigma^2}\left(-2(\mathbf{y} - \mathbf{X}\boldsymbol{\theta})^\top \mathbf{X}\right) = \frac{1}{\sigma^2}\left(-\mathbf{y}^\top \mathbf{X} + \boldsymbol{\theta}^\top \mathbf{X}^\top \mathbf{X}\right) = 0$$

$$\iff \boldsymbol{\theta}_{\mathsf{ML}}^\top \mathbf{X}^\top \mathbf{X} = \mathbf{y}^\top \mathbf{X}$$

$$\iff \boldsymbol{\theta}_{\mathsf{ML}}^\top = \mathbf{y}^\top \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \quad (\mathbf{X}^\top \mathbf{X} \text{ is positive definite if } \mathrm{rk}(\mathbf{X}) = D)$$

$$\iff \boldsymbol{\theta}_{\mathsf{ML}} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$$

MLE with Features

Linear regression: Linear in terms of the parameters.

Linear regression with the parameter $\boldsymbol{\theta} \in \mathbb{R}^K$, $\phi(\mathbf{x}) : \mathbb{R}^D \mapsto \mathbb{R}^K$:

$$p(y \mid \mathbf{x}) = \mathcal{N}(y \mid \phi(\mathbf{x})\boldsymbol{\theta}, \sigma^2) \iff y = \phi(\mathbf{x})\boldsymbol{\theta} + \epsilon = \sum_{k=0}^{K-1} \theta_k \phi_k(\mathbf{x}) + \epsilon$$

(Example) Polynomial regression. For $x \in \mathbb{R}$ and $\boldsymbol{\theta} \in \mathbb{R}^K$, we lift the original 1-D input into K-D feature space with monomials x^k:

$$\phi(x) = \begin{pmatrix} \phi_0(x) \\ \vdots \\ \phi_{K-1}(x) \end{pmatrix} = \begin{pmatrix} 1 \\ \vdots \\ x^{K-1} \end{pmatrix} \in \mathbb{R}^K \implies f(x) = \sum_{k=0}^{K-1} \theta_k x^k$$

Feature matrix and MLE

The entire training set $\{\mathbf{x}_1, \dots, \mathbf{x}_N\}$,

$$\boldsymbol{\Phi} \stackrel{\text{def}}{=} \begin{pmatrix} \phi(\mathbf{x}_1) \\ \vdots \\ \phi(\mathbf{x}_N) \end{pmatrix} = \begin{pmatrix} \phi_0(\mathbf{x}_1) & \cdots & \phi_{K-1}(\mathbf{x}_1) \\ \vdots & \ddots & \vdots \\ \phi_0(\mathbf{x}_N) & \cdots & \phi_{K-1}(\mathbf{x}_N) \end{pmatrix} \in \mathbb{R}^{N \times K},$$

$\boldsymbol{\Phi}_{ij} = \phi_j(\mathbf{x}_i)$, $\phi_j : \mathbb{R}^D \mapsto \mathbb{R}$.

Negative-log likelihood for $f(\mathbf{x}) = \phi^\intercal(\mathbf{x})\boldsymbol{\theta} + \mathcal{N}(0, \sigma^2)$:

$$-\log p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta}) = \frac{1}{2\sigma^2}\|\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}\|^2 + \text{const} \qquad p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta}) = \mathcal{N}(\mathbf{y} \mid \boldsymbol{\Phi}\boldsymbol{\theta}, \sigma^2\mathbf{I})$$

MLE:

$$\boldsymbol{\theta}_{\text{ML}} = (\boldsymbol{\Phi}^\top\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}\mathbf{y}$$

Overview

1. Problem formulation.
2. Parameters estimation: Maximum Likelihood (ML).
3. Parameters estimation: Maximum A Posteriori.
4. Bayesian linear regression.
5. Maximum Likelihood as Orthogonal Projection.

MAPE (Maximum A Posteriori Estimation)

MLE: prone to overfitting, where the magnitude of the parameters becomes large.

A prior distribution $p(\boldsymbol{\theta})$ expresses which values of $\boldsymbol{\theta}$ are plausible.

MAPE and Bayes' theorem:

$$p(\boldsymbol{\theta} \mid \mathcal{X}, \mathcal{Y}) = \frac{p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta}) p(\boldsymbol{\theta})}{p(\mathcal{Y} \mid \mathcal{X})} \implies \boldsymbol{\theta}_{\text{MAP}} \in \arg\min_{\boldsymbol{\theta}} \Big(-\log p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta}) - \log p(\boldsymbol{\theta}) \Big)$$

Gradient:

$$\frac{\mathrm{d} \log p(\boldsymbol{\theta} | \mathcal{X}, \mathcal{Y})}{\mathrm{d}\boldsymbol{\theta}} = -\frac{\mathrm{d} \log p(\mathcal{Y} | \mathcal{X}, \boldsymbol{\theta})}{\mathrm{d}\boldsymbol{\theta}} - \frac{\mathrm{d} \log p(\boldsymbol{\theta})}{\mathrm{d}\boldsymbol{\theta}}$$

MAPE for Gausssian Prior

(Example) A (conjugate) Gaussian prior $p(\boldsymbol{\theta}) \sim \mathcal{N}(\mathbf{0}, b^2\mathbf{I})$.

For Gaussian likelihood, Gaussian prior $\implies$ Gaussian posterior.

Negative-log posterior for $f(\mathbf{x}) = \phi^\mathsf{T}(\mathbf{x})\boldsymbol{\theta} + \mathcal{N}(0, \sigma^2)$ and $p(\boldsymbol{\theta}) \sim \mathcal{N}(\mathbf{0}, b^2\mathbf{I})$:

$$-\log p(\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y}) = \frac{1}{2\sigma^2}(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta})^\mathsf{T}(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}) + \frac{1}{2b^2}\boldsymbol{\theta}^\mathsf{T}\boldsymbol{\theta} + \text{const}$$

Gradient

$$-\frac{\mathrm{d}\log p(\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y})}{\mathrm{d}\boldsymbol{\theta}} = \frac{1}{\sigma^2}(\boldsymbol{\theta}^\mathsf{T}\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi} - \mathbf{y}^\mathsf{T}\boldsymbol{\Phi}) + \frac{1}{b^2}\boldsymbol{\theta}^\mathsf{T}$$

MAPE for Gausssian Prior

MAP vs. ML

$$\boldsymbol{\theta}_{\text{MAP}} = \underbrace{\left(\boldsymbol{\Phi}^\top\boldsymbol{\Phi} + \frac{\sigma^2}{b^2}\mathbf{I}\right)}_{(*)}^{-1}\boldsymbol{\Phi}^\top\mathbf{y}, \quad \boldsymbol{\theta}_{\text{ML}} = (\boldsymbol{\Phi}^\top\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^\top\mathbf{y}$$

The term $\frac{\sigma^2}{b^2}\mathbf{I}$:

- Ensures that $(*)$ is symmetric and strictly positive definite.
- Acts as a **regularizer.**

MAPE for general Gausssian prior

(Example) A (conjugate) Gaussian prior $p(\boldsymbol{\theta}) \sim \mathcal{N}(\mathbf{m}_0, \mathbf{S}_0)$

Negative-log posterior for $f(\mathbf{x}) = \phi^\intercal(\mathbf{x})\boldsymbol{\theta} + \mathcal{N}(0, \sigma^2)$ and $p(\boldsymbol{\theta}) \sim \mathcal{N}(\mathbf{m}_0, \mathbf{S}_0)$:

$$-\log p(\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y}) = \frac{1}{2\sigma^2}(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta})^\intercal(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}) + \frac{1}{2}(\boldsymbol{\theta} - \mathbf{m}_0)^\intercal \mathbf{S}_0^{-1}(\boldsymbol{\theta} - \mathbf{m}_0) + \text{const}$$

We will use this later for computing the parameter posterior distribution in Bayesian linear regression.

MAPE versus explicit regularization

Explicit regularizer in regularized least squares (RLS).

$$\|\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}\|^2 + \lambda\|\boldsymbol{\theta}\|^2$$

MAPE with Gaussian prior $p(\boldsymbol{\theta}) \sim \mathcal{N}(\mathbf{0}, b^2\mathbf{I})$

Negative log-Gaussian prior:

$$-\log p(\boldsymbol{\theta}) = \frac{1}{2b^2}\boldsymbol{\theta}^\mathsf{T}\boldsymbol{\theta} + \text{const},$$

where $\lambda = 1/2b^2$ is the regularization term.

Not surprising that we have $\boldsymbol{\theta}_{\text{RLS}} = \left(\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi} + \lambda\mathbf{I}\right)^{-1}\boldsymbol{\Phi}^\mathsf{T}\mathbf{y}$.

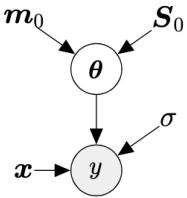
Overview

1. Problem formulation.
2. Parameters estimation: Maximum Likelihood (ML).
3. Parameters estimation: Maximum A Posteriori.
4. Bayesian linear regression.
5. Maximum Likelihood as Orthogonal Projection.

Bayesian linear regression

Model:

$$\begin{aligned} \text{prior} \quad & p(\boldsymbol{\theta}) \sim \mathcal{N}(\mathbf{m}_0, \mathbf{S}_0) \\ \text{likelihood} \quad & p(y|\mathbf{x}, \boldsymbol{\theta}) \sim \mathcal{N}(y \mid \phi^\intercal(\mathbf{x})\boldsymbol{\theta}, \sigma^2) \\ \text{joint} \quad & p(y, \boldsymbol{\theta}|\mathbf{x}) = p(y \mid \mathbf{x}, \boldsymbol{\theta})p(\boldsymbol{\theta}) \end{aligned}$$



Goal: For an input $\mathbf{x}_*$, we want to compute the following posterior predictive distribution of y_*:

$$p(y_*|x_*, \mathcal{X}, \mathcal{Y}) = \int \overbrace{p(y_*|\mathbf{x}_*, \boldsymbol{\theta})}^{\text{likelihood}} \overbrace{p(\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y})}^{(*)} \, \mathrm{d}\boldsymbol{\theta}$$

$(*)$: parameter posterior distribution that needs to be computed.

Parameter posterior distribution

Parameter posterior distribution

$$p(\boldsymbol{\theta} \mid \mathcal{X}, \mathcal{Y}) = \mathcal{N}(\boldsymbol{\theta} \mid \mathbf{m}_N, \mathbf{S}_N), \quad \text{where}$$

$$\mathbf{S}_N = (\mathbf{S}_0^{-1} + \sigma^2 \boldsymbol{\Phi}^\intercal \boldsymbol{\Phi})^{-1}, \quad \mathbf{m}_N = \mathbf{S}_N(\mathbf{S}_0^{-1}\mathbf{m}_0 + \sigma^{-2}\boldsymbol{\Phi}^\intercal \mathbf{y})$$

(Proof Sketch)

From the negative-log posterior for general Gaussian prior,

$$-\log p(\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y}) = \frac{1}{2\sigma^2}(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta})^\intercal(\mathbf{y} - \boldsymbol{\Phi}\boldsymbol{\theta}) + \frac{1}{2}(\boldsymbol{\theta} - \mathbf{m}_0)^\intercal \mathbf{S}_0^{-1}(\boldsymbol{\theta} - \mathbf{m}_0) + \text{const}$$

Parameter posterior distribution

$$= \frac{1}{2}\Big(\sigma^{-2}\mathbf{y}^\intercal\mathbf{y} - 2\sigma^{-2}\mathbf{y}^\intercal\boldsymbol{\Phi}\boldsymbol{\theta} + \boldsymbol{\theta}^\intercal\sigma^{-2}\boldsymbol{\Phi}^\intercal\boldsymbol{\Phi}\boldsymbol{\theta} + \boldsymbol{\theta}^\intercal\mathbf{S}_0^{-1}\boldsymbol{\theta} - 2\mathbf{m}_0^\intercal\mathbf{S}_0^{-1}\boldsymbol{\theta} + \mathbf{m}_0^\intercal\mathbf{S}_0^{-1}\mathbf{m}_0\Big)$$

$$= \frac{1}{2}\Big(\boldsymbol{\theta}^\intercal(\sigma^{-2}\boldsymbol{\Phi}^\intercal\boldsymbol{\Phi} + \mathbf{S}_0^{-1})\boldsymbol{\theta} - 2(\sigma^{-2}\boldsymbol{\Phi}^\intercal\mathbf{y} + \mathbf{S}_0^{-1}\mathbf{m}_0)^\intercal\boldsymbol{\theta}\Big) + \text{const}$$

cyan color: quadratic term, orange color: linear term.

$p(\boldsymbol{\theta} \mid \mathcal{X}, \mathcal{Y}) \propto \exp(\text{ quadratic in } \boldsymbol{\theta}\ ) \implies$ Gaussian distribution.

Parameter posterior distribution

Assume that $p(\boldsymbol{\theta} \mid \mathcal{X}, \mathcal{Y}) = \mathcal{N}(\boldsymbol{\theta} \mid \mathbf{m}_N, \mathbf{S}_N)$, and find $\mathbf{m}_N$ and $\mathbf{S}_N$.

$$
\begin{aligned}
-\log \mathcal{N}(\boldsymbol{\theta} \mid \mathbf{m}_N, \mathbf{S}_N) &= \frac{1}{2}(\boldsymbol{\theta} - \mathbf{m}_N)^\intercal \mathbf{S}_N^{-1}(\boldsymbol{\theta} - \mathbf{m}_N) + \text{const} \\
&= \frac{1}{2}\left(\boldsymbol{\theta}^\intercal \mathbf{S}_N^{-1}\boldsymbol{\theta} - 2\mathbf{m}_N{}^\intercal \mathbf{S}_N^{-1}\boldsymbol{\theta} + \mathbf{m}_N^\intercal \mathbf{S}_N^{-1}\mathbf{m}_N\right) + \text{const}
\end{aligned}
$$

Thus, $\mathbf{S}_N^{-1} = \sigma^{-2}\boldsymbol{\Phi}^\intercal\boldsymbol{\Phi} + \mathbf{S}_0^{-1}$ and $\mathbf{m}_N^\intercal \mathbf{S}_N^{-1} = (\sigma^{-2}\boldsymbol{\Phi}^\intercal\mathbf{y} + \mathbf{S}_0^{-1}\mathbf{m}_0)^\intercal$.

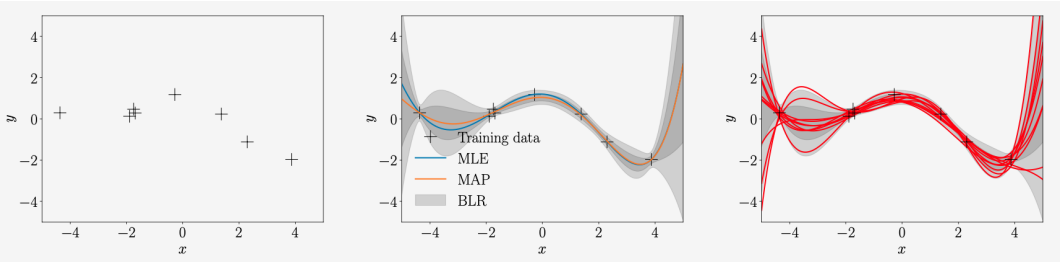
Posterior predictions

Posterior predictive distribution

$$
\begin{aligned}
p(y_*|x_*, \mathcal{X}, \mathcal{Y}) &= \int p(y_*|\mathbf{x}_*, \boldsymbol{\theta}) p(\boldsymbol{\theta}|\mathcal{X}, \mathcal{Y}) \mathrm{d}\boldsymbol{\theta} \\
&= \int \mathcal{N}\Big(y_*|\phi^\mathsf{T}(\mathbf{x}_*)\boldsymbol{\theta}, \sigma^2\Big) \mathcal{N}\Big(\boldsymbol{\theta}|\mathbf{m}_N, \mathbf{S}_N\Big) \mathrm{d}\boldsymbol{\theta} \\
&= \mathcal{N}\Big(y_*|\phi^\mathsf{T}(\mathbf{x}_*)\mathbf{m}_N, \phi^\mathsf{T}(\mathbf{x}_*)\mathbf{S}_N\phi(\mathbf{x}_*) + \sigma^2\Big)
\end{aligned}
$$

The mean $\phi^\mathsf{T}(\mathbf{x}_*)\mathbf{m}_N$ coincides with the MAP estimate.

Posterior predictions



Computing marginal likelihood

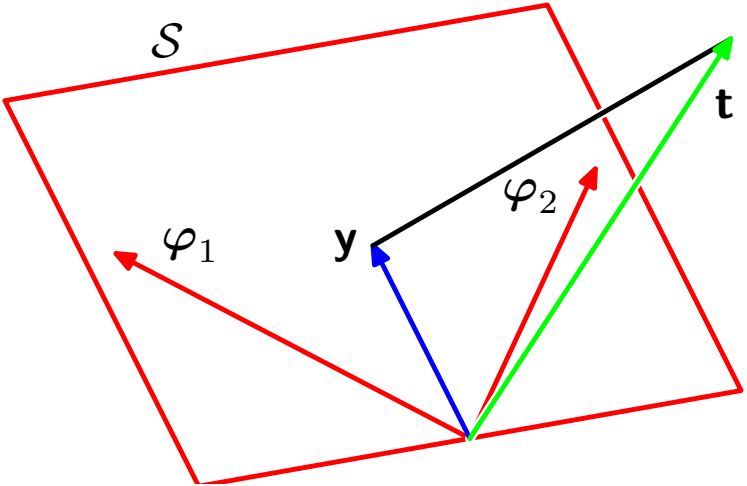
Likelihood: $p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta})$, Marginal likelihood:
$p(\mathcal{Y} \mid \mathcal{X}) = \int p(\mathcal{Y} \mid \mathcal{X}, \boldsymbol{\theta}) p(\boldsymbol{\theta}) \mathsf{d}\boldsymbol{\theta}$

$$
\begin{aligned}
p(\mathcal{Y}|\mathcal{X}) &= \int p(\mathcal{Y}|\mathcal{X}, \boldsymbol{\theta}) p(\boldsymbol{\theta}) \mathsf{d}\boldsymbol{\theta} = \int \mathcal{N}(\mathbf{y}|\boldsymbol{\Phi}\boldsymbol{\theta}, \sigma^2\mathbf{I}) \mathcal{N}(\boldsymbol{\theta}|\mathbf{m}_0, \mathbf{S}_0) \, \mathsf{d}\boldsymbol{\theta} \\
&= \mathcal{N}(\mathbf{y} \mid \boldsymbol{\Phi}\mathbf{m}_0, \boldsymbol{\Phi}\mathbf{S}_0\boldsymbol{\Phi}^\intercal + \sigma^2\mathbf{I})
\end{aligned}
$$

Overview

1. Problem formulation.
2. Parameters estimation: Maximum Likelihood (ML).
3. Parameters estimation: Maximum A Posteriori.
4. Bayesian linear regression.
5. Maximum Likelihood as Orthogonal Projection.

ML as orthogonal projection



ML as orthogonal projection: linear case

Assume a single-feature linear model with Gaussian noise:

$$f(x_i) = x_i\theta + \mathcal{N}(0, \sigma^2),$$

and let $\mathbf{x} \in \mathbb{R}^n$ be the vector of inputs, $\mathbf{x} = (x_1, \dots, x_n)^\top$, and $\mathbf{y} \in \mathbb{R}^n$ the vector of outputs.

The ML (least squares) estimator is:

$$\theta_{\text{ML}} = \frac{\mathbf{x}^\top \mathbf{y}}{\mathbf{x}^\top \mathbf{x}} \in \mathbb{R}.$$

The prediction vector becomes:

$$\hat{\mathbf{y}} = \mathbf{x}\,\theta_{\text{ML}} = \frac{\mathbf{x}\mathbf{x}^\top}{\mathbf{x}^\top \mathbf{x}}\,\mathbf{y}.$$

Nonlinear case (feature mapping)

Now consider a (possibly) nonlinear model implemented via a feature map:

$$f(\mathbf{x}_i) = \phi(\mathbf{x}_i)^\top \boldsymbol{\theta} + \mathcal{N}(0, \sigma^2),$$

where $\phi(\mathbf{x}_i) \in \mathbb{R}^K$ is the feature vector for sample i, $\boldsymbol{\theta} \in \mathbb{R}^K$ are the parameters.

Collect all features in the design matrix $\boldsymbol{\Phi} \in \mathbb{R}^{n \times K}$, where the i-th row is $\phi(\mathbf{x}_i)^\top$.

The ML estimator is:

$$\boldsymbol{\theta}_{\text{ML}} = (\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \mathbf{y}.$$

The prediction vector becomes:

$$\hat{\mathbf{y}} = \boldsymbol{\Phi} \boldsymbol{\theta}_{\text{ML}} = \boldsymbol{\Phi}(\boldsymbol{\Phi}^\top \boldsymbol{\Phi})^{-1} \boldsymbol{\Phi}^\top \mathbf{y}.$$

Interpretation: $\hat{\mathbf{y}}$ is the orthogonal projection of $\mathbf{y}$ onto the K-dimensional subspace $\text{col}(\boldsymbol{\Phi})$ spanned by the columns of $\boldsymbol{\Phi}$.

Summary and other issues

Linear regression for Gaussian likelihood and conjugate Gaussian priors. Nice analytical results and closed forms.

Other forms of likelihoods for other applications (e.g., classification).

GLM (generalized linear model): $y = \sigma \circ f$ (σ: activation function).

- No longer linear in $\boldsymbol{\theta}$.
- Logistic regression: $\sigma(f) = \dfrac{1}{1 + \exp(-f)} \in [0, 1]$ (interpreted as the probability of becoming 1).
- Building blocks of (deep) feedforward neural nets.

$\mathbf{y} = \sigma(\mathbf{A}\mathbf{x} + \mathbf{b})$. $\mathbf{A}$: weight matrix, $\mathbf{b}$: bias vector.

K-layer deep neural nets: $\mathbf{x}_{k+1} = f_k(\mathbf{x}_k)$, $f_k(\mathbf{x}_k) = \sigma_k(\mathbf{A}_k\mathbf{x}_k + \mathbf{b}_k)$.

Summary and other issues

A distribution over parameters $\longrightarrow$ a distribution over functions.

Gaussian process: distribution over functions without detouring via parameters.

Closely related to BLR and support vector regression, also interpreted as Bayesian neural network with a single hidden layer and the infinite number of units.

Gaussian likelihood, but non-Gaussian prior:

- When $N \ll D$ (small training data).
- Prior that enforces sparsity, e.g., Laplace prior.
- A linear regression with the Laplace prior $=$ linear regression with LASSO (L1 regularization).